

Reflections App

Architecture Documentation

*LangGraph-Orchestrated Multi-Agent System
for Guided Student Reflection*

May 2026

Table of Contents

1.	Executive Summary	3
2.	System Architecture Overview	4
3.	LangGraph Integration — Deep Dive	6
	3a. Shared State: ReflectionState	6
	3b. Init Graph	7
	3c. Message Graph	8
	3d. End Graph	9
	3e. PostgreSQL Checkpointing	9
4.	Multi-Model Agentic Architecture	10
	4a. Agent Roles	10
	4b. LLM Provider Abstraction	11
	4c. LangSmith Tracing	11
	4d. LLM Response Protocol	12
5.	Session Lifecycle Walkthrough	13
6.	Adaptive Pedagogy System	14
7.	CLI Agent Architecture	15
8.	Data Model	17
9.	Deployment Architecture	18

1. Executive Summary

Reflections App is an AI-powered academic reflection chatbot designed for higher education. Professors configure learning modules with topics, depth levels, and probing styles. Students engage with an AI tutor that guides Socratic reflection on course material. Transcripts are automatically evaluated across six dimensions and stored for analytics.

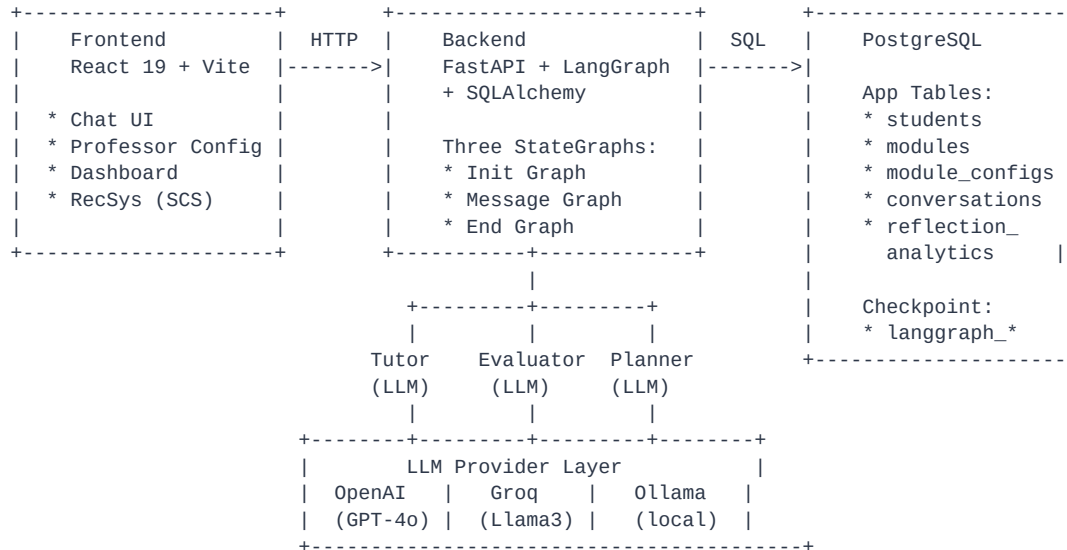
Key Architectural Decisions

- **LangGraph StateGraphs** for stateful, multi-step agent orchestration with PostgreSQL checkpointing — enabling stateless HTTP endpoints that automatically recover session state
- **Multi-model LLM routing** — three specialized agent roles (Tutor, Evaluator, Planner) each independently configurable across OpenAI, Groq, and Ollama providers via environment variables
- **LangSmith tracing** for end-to-end observability across all LLM calls, with automatic token usage tracking and latency measurement
- **Two implementations**: a production FastAPI web backend with React 19 frontend, and a standalone CLI agent for development and research
- **Adaptive pedagogy** — the system tracks weak areas per student across sessions and dynamically adjusts questioning strategy with a bonus phase targeting misconceptions

2. System Architecture Overview

The system is organized in four layers: a React frontend, a FastAPI backend with LangGraph agent orchestration, a PostgreSQL database for persistence and checkpointing, and a pluggable LLM provider layer supporting multiple model providers.

High-Level Architecture



Layer Descriptions

- **Frontend:** React 19 with Vite bundler and React Router. Four pages: Chat (student reflection interface supporting topic-based and milestone-based modules), ProfessorConfig (module parameter configuration), Dashboard (analytics visualization), and RecSys (similarity-based recommendation system).
- **Backend:** FastAPI with SQLAlchemy ORM. Three LangGraph StateGraphs orchestrate the chat session lifecycle. Each graph is compiled with a shared PostgreSQL checkpointer for stateful session recovery across HTTP requests.
- **Database:** PostgreSQL stores both application data (students, modules, conversations, analytics) and LangGraph checkpoint tables for session state persistence. Compatible with Supabase PgBouncer pooler.
- **LLM Layer:** Three providers (OpenAI, Groq, Ollama) accessible via an abstract factory pattern. Each agent role (Tutor, Evaluator, Planner) can be independently assigned to any provider/model combination through environment variable overrides.

3. LangGraph Integration — Deep Dive

LangGraph is the core orchestration framework. The backend decomposes the chat session lifecycle into three separate StateGraphs, each handling a distinct phase. All graphs share a common ReflectionState TypedDict and are compiled with a PostgreSQL-backed checkpointer for stateful session recovery.

3a. Shared State: ReflectionState

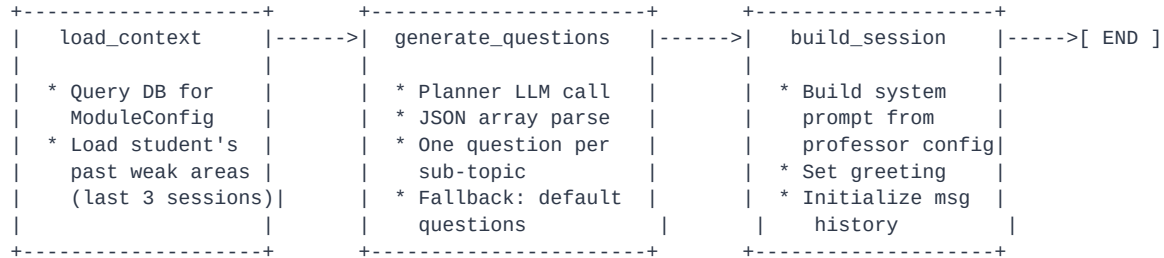
All three backend graphs share the ReflectionState TypedDict, which contains 23 fields organized into six categories. This state is automatically persisted and recovered via the PostgreSQL checkpointer using the session_id as the LangGraph thread_id.

Field	Type	Category	Purpose
session_id	str	Identity	Unique session ID, used as LangGraph thread_id
student_id	str	Identity	Student reference
module_id	str	Identity	Learning module reference
config	dict	Config	Professor configuration (topics, depth, style)
module_type	str	Config	"topic_based" or "milestone_based"
past_weak_areas	list[str]	Config	Cross-session weak topic tracking (last 3 sessions)
questions	list[str]	Questions	Pre-generated questions (one per sub-topic)
sub_topics	list[str]	Questions	Sub-topics from module configuration
question_idx	int	Questions	Current question progress pointer
is_bonus_phase	bool	Questions	Whether adaptive bonus phase is active
wrong_subtopics	list[str]	Questions	Topics with inadequate student answers
messages	list[dict]	Conversation	Full message history [{role, content}]
system_prompt	str	Conversation	Generated system prompt with professor rules
user_message	str	Turn I/O	Current turn's raw user input
current_reply	str	Turn I/O	Current turn's parsed LLM reply
adequate	bool	Turn I/O	Whether current answer was judged adequate
bonus_just_started	bool	Turn I/O	Transition flag for bonus phase
phase	str	Lifecycle	"init", "active", "bonus_transition", or "ended"
greeting	str	Lifecycle	Initial greeting / first question
total_questions	int	Lifecycle	Total pre-generated question count

Field	Type	Category	Purpose
transcript	str	Post-session	Formatted transcript (populated at /end)
evaluation	dict?	Post-session	Evaluation results (populated at /end)
conversation_id	str?	Post-session	Database conversation ID (populated at /end)

3b. Init Graph (POST /api/chat/start)

The Init Graph runs once when a student starts a new session. It loads the professor's module configuration from PostgreSQL, retrieves past weak areas from up to 3 prior sessions, calls the Planner LLM to generate one question per sub-topic, then builds the full system prompt and initial message history.

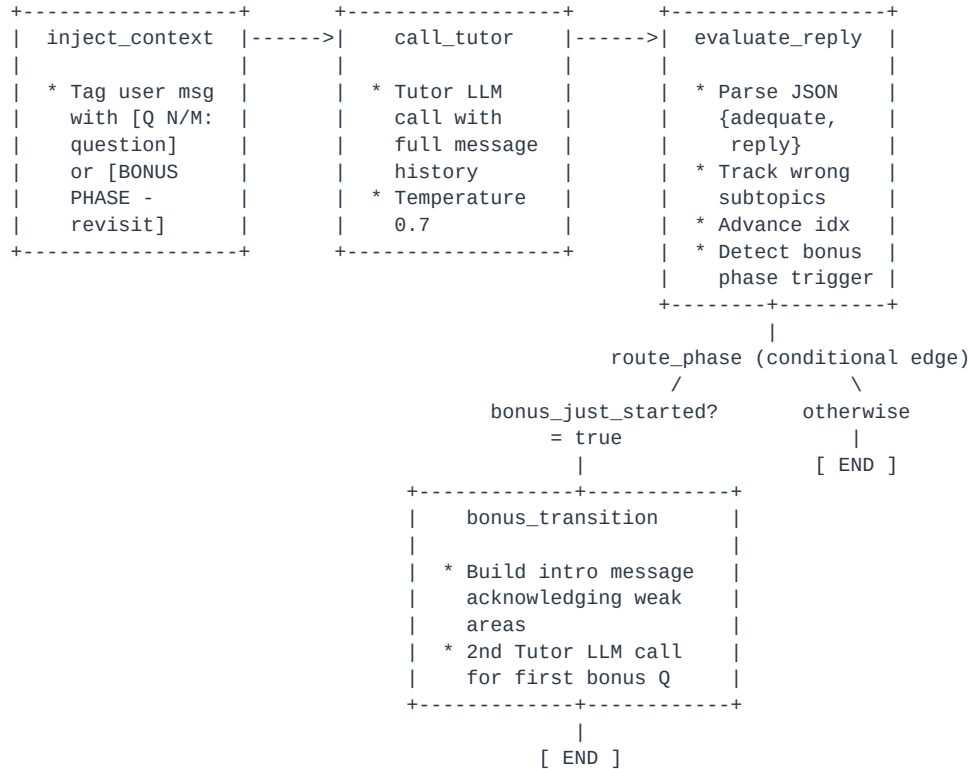


Node Details:

- **load_context** — Queries the ModuleConfig table for the professor's configuration and loads the student's past weak areas from the 3 most recent ReflectionAnalytics records. Receives the database session via RunnableConfig injection.
- **generate_questions** — Calls the Planner LLM with the sub-topics list to generate one focused question per topic. Parses the JSON array response with a fallback to default questions if parsing fails.
- **build_session** — Constructs the system prompt incorporating professor rules (expected_depth, probing_style, custom_notes), initializes the message history with the system prompt, and sets the greeting message.

3c. Message Graph (POST /api/chat/message)

This is the core per-turn processing graph. Each student message flows through context injection, Tutor LLM generation, and Evaluator parsing. A conditional edge (`route_phase`) determines whether to trigger the bonus phase transition.



Conditional Routing:

The **route_phase** function inspects the `bonus_just_started` flag. When a student completes all main questions (`question_idx >= total_questions`), the `evaluate_reply` node sets this flag to `True`, routing to the `bonus_transition` node. Otherwise, the graph terminates and returns the reply to the API caller.

The **bonus_transition** node builds an introductory message acknowledging the student's weak areas, then fires a second Tutor LLM call to generate the first bonus question targeting those specific subtopics.

3d. End Graph (POST /api/chat/end)

The End Graph runs once when a session concludes. It formats the conversation transcript and runs a comprehensive evaluation across six dimensions.

+-----+ format_transcript -----> evaluate_session ----->[END] +-----+	+-----+ * Strip system msgs * Join turns as plain text * Persist to Conversation table in DB +-----+	+-----+ * Evaluator LLM scores transcript * Six dimensions: topics_covered, missing_topics, misconceptions, depth_score, confidence_level, engagement_score * Persist to ReflectionAnalytics +-----+
--	---	--

3e. PostgreSQL Checkpointing

- Uses PostgresSaver from `langgraph.checkpoint.postgres` for production-grade state persistence
- Thread ID equals `session_id` — each student session gets its own isolated state checkpoint
- Checkpoint tables auto-created at app startup via `_checkpointer.setup()`
- All three graphs compiled with the shared checkpointer instance for consistent state access
- Connection uses `prepare_threshold=0` for Supabase PgBouncer transaction-mode pooler compatibility
- Enables fully stateless HTTP endpoints — each `/message` call automatically loads prior state from the checkpoint via the `thread_id`, with no explicit state management in the API layer

4. Multi-Model Agentic Architecture

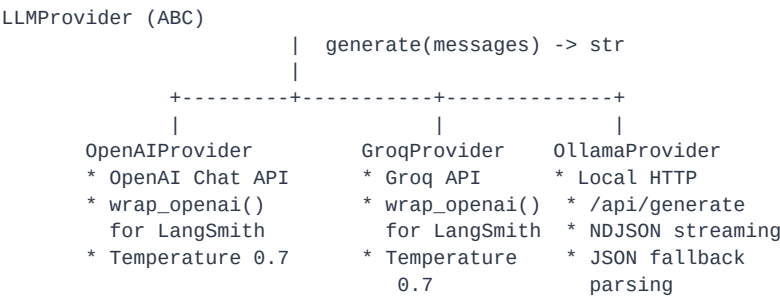
The system implements a multi-agent architecture where each agent role is backed by an independently configurable LLM. This enables cost optimization (cheaper models for simple tasks), latency tuning (faster providers for non-critical paths), and experimentation (swapping models without code changes).

4a. Agent Roles

Role	Purpose	Graph Nodes	Default Model
Tutor	Generates conversational responses; evaluates answer adequacy via JSON format	call_tutor, bonus_transition	gpt-4o-mini
Evaluator	Judges per-turn adequacy; produces post-session analytics across 6 dimensions	evaluate_reply, evaluate_session	gpt-4o-mini
Planner	Generates questions from sub-topics; routes phase transitions	generate_questions, route_phase, bonus_transition	gpt-4o-mini

Each role can be independently configured to use a different LLM provider and model via environment variables, enabling heterogeneous deployments.

4b. LLM Provider Abstraction



Resolution Order (get_llm_for_role):

1. Check {ROLE}_LLM_PROVIDER (e.g., TUTOR_LLM_PROVIDER=openai)
2. Fall back to global LLM_PROVIDER (default: openai)
3. Check {ROLE}_{PROVIDER}_MODEL (e.g., TUTOR_OPENAI_MODEL=gpt-4o)
4. Fall back to {PROVIDER}_MODEL (e.g., OPENAI_MODEL=gpt-4o-mini)

Example Heterogeneous Configuration:

Agent Role	Provider	Model	Rationale
Tutor	OpenAI	gpt-4o	High quality for student-facing responses
Evaluator	OpenAI	gpt-4o-mini	Cost-effective for structured JSON parsing
Planner	Groq	llama3-8b-8192	Fast question generation, lower cost

4c. LangSmith Tracing

LangSmith observability is integrated via `wrap_openai()` from the `langsmith` package, applied to both OpenAI and Groq provider clients. This provides:

- Automatic trace emission for all LLM calls across Tutor, Evaluator, and Planner agents
- Token usage tracking per call for cost monitoring
- Model and temperature parameter logging for reproducibility
- Latency measurements for performance optimization

Activated when `LANGSMITH_TRACING=true` and `LANGSMITH_API_KEY` are set in the environment.

4d. LLM Response Protocol

The Tutor LLM outputs strict JSON for every response:

```
{"adequate": true/false, "reply": "plain text response"}
```

The **evaluate_reply** node parses this JSON to extract the student-facing reply text and determine whether the answer was adequate. Adequate responses advance the question index; inadequate answers add the current sub-topic to `wrong_subtopics` for later review in the bonus phase. If JSON parsing fails, adequate defaults to false and the raw text is used.

5. Session Lifecycle Walkthrough

The complete session lifecycle spans four phases, each mapped to a specific API endpoint and LangGraph invocation.

1. PROFESSOR CONFIGURES MODULE

POST /api/modules/{id}/config

```
+-----+
| required_topics, sub_topics,          |
| expected_depth (surface|applied|      |
| analytical), probing_style            |
| (supportive|socratic)                 |
+-----+
```

|
v

2. STUDENT STARTS SESSION

POST /api/chat/start --> Init Graph

```
+-----+
| Response: {                           |
|   session_id: "uuid",                 |
|   greeting: "Welcome! Let's reflect..", |
|   total_questions: 5                  |
| }                                     |
+-----+
```

|
v

3. STUDENT SENDS MESSAGES (LOOP)

POST /api/chat/message --> Message Graph

```
+-----+
| Request: { session_id, message }      |
| Response: {                           |
|   reply: "That's interesting...",     |
|   question_index: 3,                  |
|   total_questions: 5,                 |
|   is_bonus_phase: false               |
| }                                     |
|                                       |
| Internal: inject_context > call_tutor > |
| evaluate_reply > [bonus_transition]    |
+-----+
```

|
v

4. SESSION ENDS

POST /api/chat/end --> End Graph

```
+-----+
| Response: {                           |
|   conversation_id: "uuid",            |
|   evaluation: {                       |
|     topics_covered: [...],            |
|     missing_topics: [...],            |
|     misconceptions: [...],            |
|     reflection_depth_score: 0.78,     |
|     confidence_level: 4,              |
|     engagement_score: 0.85            |
|   }                                   |
| }                                     |
+-----+
```

6. Adaptive Pedagogy System

The system implements an adaptive questioning strategy that tracks student performance and dynamically adjusts the conversation flow.

Main Question Phase

- Questions are asked sequentially from the pre-generated list (one per sub-topic)
- Each answer is evaluated by the Tutor LLM: adequate responses advance to the next question; inadequate answers trigger a follow-up on the same topic
- Wrong sub-topics are tracked in the `wrong_subtopics` list for later review

Bonus Phase Trigger

- Activated when `question_idx >= total_questions` (all main questions have been answered)
- `is_bonus_phase` flips to true; `bonus_just_started` is set for exactly one turn
- The conditional edge in the Message Graph routes to the `bonus_transition` node

Bonus Transition

- The system generates an introductory message acknowledging the student's weak areas
- A second Tutor LLM call generates the first bonus question targeting `wrong_subtopics`
- Context tag changes from `[Q N/M: question]` to `[BONUS PHASE - revisit: topic1, topic2]`

Cross-Session Learning

- Up to 3 prior sessions' `missing_topics` loaded as `past_weak_areas` during Init Graph execution
- Injected into the system prompt so the tutor can proactively address recurring gaps
- Enables longitudinal improvement tracking across multiple sessions

7. CLI Agent Architecture

The CLI agent provides a standalone implementation using a single, comprehensive 7-node LangGraph with conditional loops. It is designed for development, research, and offline use without requiring the full web stack.

[START]



+-----+
|
[END]

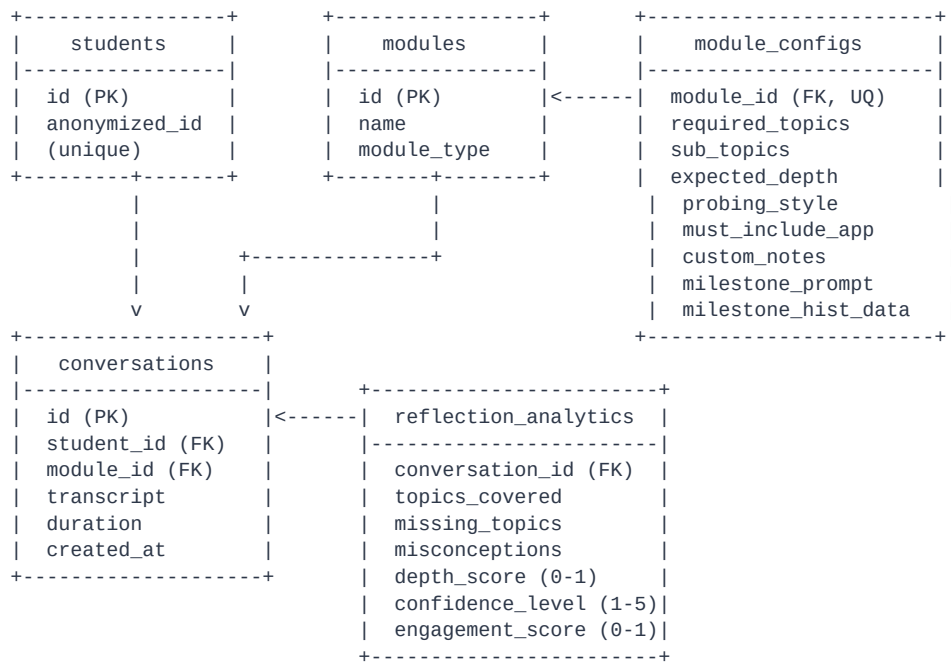
CLI vs Backend Comparison

Aspect	Backend (Web)	CLI Agent
Graph count	3 separate graphs	1 unified graph
State type	TypedDict (ReflectionState)	Pydantic (SessionState)
Checkpointing	PostgresSaver (production)	None (single-run)
Question generation	Pre-generated all at once	Dynamic per-turn via Planner
Adaptive behavior	Bonus phase on weak areas	Stall detection + auto-pivot
Conversation phases	init, active, bonus_transition, ended	warmup, probing, synthesis, finalize
Persistence	PostgreSQL tables	JSON/Markdown files
LLM framework	Custom LLMProvider ABC	LangChain ChatOpenAI
Agenda source	Module config from DB	YAML agenda files

8. Data Model

The application uses SQLAlchemy ORM with five core tables. LangGraph checkpoint tables are auto-created by the PostgresSaver setup process.

Entity-Relationship Diagram



Plus: LangGraph checkpoint tables (auto-created by PostgresSaver.setup())

Table Descriptions

- **students** — Student records with anonymized identifiers for privacy protection
- **modules** — Learning modules created by professors, typed as `topic_based` or `milestone_based`
- **module_configs** — Professor-defined configuration for each module including topics, depth levels, probing style, and optional milestone data
- **conversations** — Complete session records linking students to modules with full transcripts and timing data
- **reflection_analytics** — LLM-generated evaluation scores per conversation across six dimensions: topics covered, missing topics, misconceptions, depth score, confidence level, and engagement score

9. Deployment Architecture

Docker Compose Topology

```
docker-compose.yml
|
+-- backend (port 8000)
|   * Python 3.11-slim
|   * FastAPI + Uvicorn
|   * LangGraph graphs compiled at startup
|   * PostgresSaver checkpointer initialized
|
+-- frontend (port 3000)
|   * Node 20 build stage + Nginx Alpine serve stage
|   * Proxies /api/* to backend:8000
|
+-- External: PostgreSQL
    * Supabase (cloud) or local Docker
    * App tables + LangGraph checkpoint tables
```

Environment Variables Reference

Variable	Default	Description
DATABASE_URL	postgresql://...localhost/reflection_chatbot	PostgreSQL connection string
LLM_PROVIDER	openai	Global default LLM provider
OPENAI_API_KEY	(required)	OpenAI API key
OPENAI_MODEL	gpt-4o-mini	Default OpenAI model
GROQ_API_KEY	(optional)	Groq API key
GROQ_MODEL	llama3-8b-8192	Default Groq model
TUTOR_LLM_PROVIDER	(falls back to LLM_PROVIDER)	Tutor-specific provider override
EVALUATOR_LLM_PROVIDER	(falls back to LLM_PROVIDER)	Evaluator-specific provider override
PLANNER_LLM_PROVIDER	(falls back to LLM_PROVIDER)	Planner-specific provider override
LANGSMITH_TRACING	true	Enable LangSmith observability
LANGSMITH_API_KEY	(required for tracing)	LangSmith API key
LANGSMITH_PROJECT	reflections-app	LangSmith project name